

A Claim-Coupled Verification Method for Hard Identification Problems

Version 0.2 — 2026-07-24

Abstract

This report describes a two-layer method for hard identification-and-construction problems. Layer A is an autonomous research arena: independent contestant agents pursue discovery under adversarial peer verification, while a sovereign daemon mediates an append-only blackboard and a sealed oracle. Layer B is a claim-coupled evidence kit: independent constructions, deterministic gates, provenance checks, ladder claim statuses, and generated verification tables consolidate an accepted result. The arena determines whether a proposed solution survives the discovery funnel; the kit determines exactly what has been checked, what is exact, and what remains a human-checked proof obligation. What ships is the fixed arena package, an exchangeable challenge template and worked `ising_lift` challenge, a deterministic end-to-end arena retest, the fixed evidence kit, an exchangeable worked evidence instance, a second instance template, and this report. The governing rule is the same in both layers: nothing certifies itself.

1 Scope

The method targets problems in which an object must be identified, an explicit construction must work on every valid input, and a sealed oracle can evaluate a classical target or reject a degenerate case. The desired output is not a fitted rule for calibration rows. It is an effective algorithm, together with an extraction that recovers the oracle and a visible boundary between numerical evidence, exact verification, and proof.

The method is neither a proof assistant nor a general test framework. It is the operational discipline connecting discovery to acceptance and acceptance to evidence. Layer A provides independent search, prediction, promotion, and peer review. Layer B provides redundant reconstruction, claim-sized gates, status control, and a document that cannot quietly drift from executed checks.

2 Problem template: the challenge package

Layer A receives one exchangeable package under `challenges/<name>/`, containing `problem.md`, `config.yaml`, and `oracle.py`. A well-posed package supplies five elements:

1. a sealed `query(payload)` oracle, or an explicit problem-only mode;
2. calibration rows containing a deliberate symmetry leak;
3. chained tasks, so a later extraction must be read from the constructed object rather than guessed independently;
4. an explicitness requirement: the finish gives a terminating algorithm for every valid input; and
5. a genericity locus with an explicit `wall` status away from it.

`config.yaml` fixes the JSON schemas, contestants, daemon address, promotion policy, and optional finish gate. Contestants see `problem.md`; only the dedicated oracle worker imports `oracle.py`. Replacing this package changes the mathematics while leaving the arena protocol unchanged.

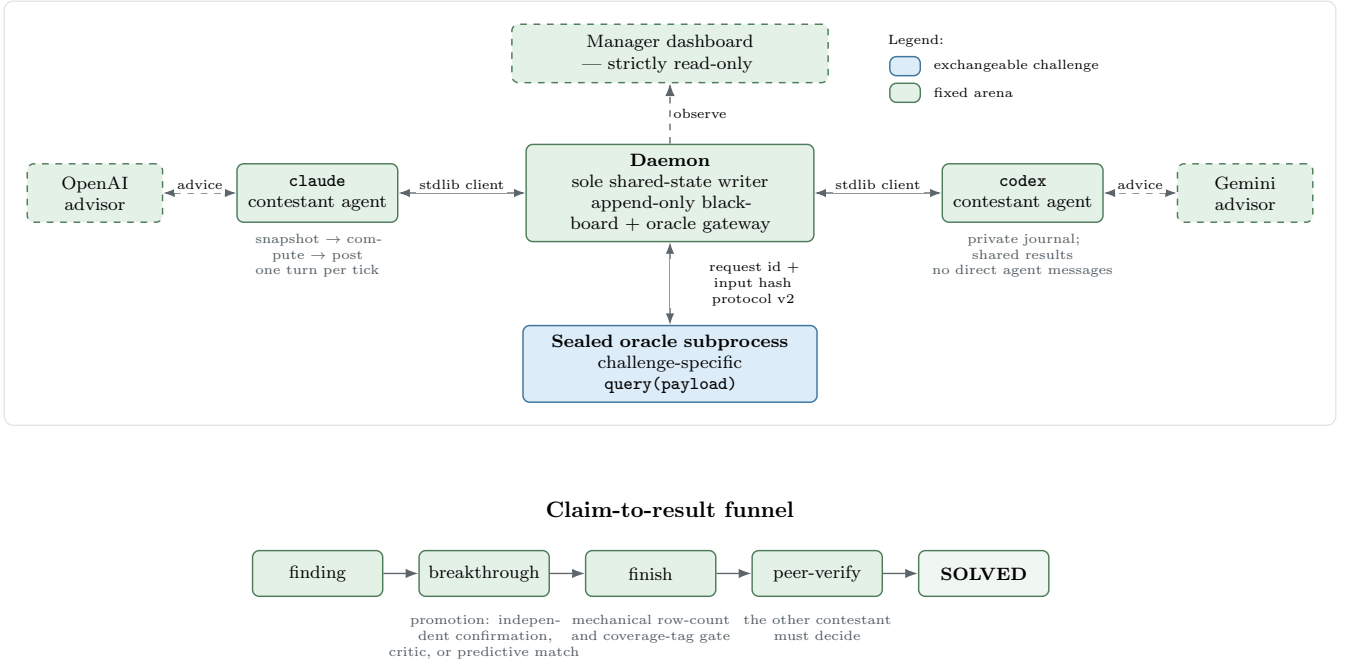


Figure 1: Layer A topology and claim funnel. The fixed daemon is sovereign over writes and oracle access; contestants coordinate through append-only state, not conversation. The blue oracle belongs to the exchangeable challenge package. A claim must cross promotion, a mechanically parsed finish gate when configured, and verification by the other contestant before the daemon writes **SOLVED**.

3 Layer A: the arena

3.1 Sovereignty and process boundaries

The daemon is the only process allowed to mutate shared state. Every contestant action is an HTTP request from the stdlib-only client; handlers serialize writes through the state manager. The daemon never imports the challenge oracle. It starts a worker subprocess, exchanges one JSON object per line, and exposes only the returned answer. This boundary prevents an ordinary import from turning the answer key into contestant context.

The seal is a process and protocol boundary, not an operating-system sandbox. An agent with unrestricted file access must still be instructed not to inspect the oracle source. The important architectural property is that normal contestant operation needs no such access and all accepted oracle evidence passes through one audited path.

3.2 Tick discipline and stigmergy

Each contestant executes the same stateless tick: pull a journal-first snapshot, compute, optionally query, append a finding or promotion candidate, update private direction and journal, append a turn record, and yield. Contestants do not direct one another. Their discoveries become environmental signals on the shared blackboard. This stigmergic structure preserves participant-level independence: routes are developed privately, while results become visible enough to support corroboration and critique.

The state split is deliberate. A contestant’s journal and current direction preserve continuity across short contexts. Shared findings, breakthroughs, finish records, and oracle logs are append-only. A round counter advances only when a turn record is posted, making missing ticks visible.

3.3 Predict before query

An oracle request may carry both a proposed output and the hypothesis that a match would support. The prediction is written before a new execution. After a correlated response, exact JSON equality determines whether it matched. When the challenge enables predictive promotion, a successful prediction can promote the hypothesis without relying on retrospective storytelling. Plain queries remain available for calibration, but prediction turns a lookup into a falsifiable test.

3.4 Request correlation, cache versions, and trust epochs

Every worker response must round-trip the protocol version, challenge name, request id, and a hash of the canonical JSON input. The runner rejects a mismatch, restarts the worker, and retries once. An executed log record also stores a cache version and an `oracle_protocol_verified` flag. Cache lookup accepts only a current-version record whose metadata matches the requested input. Older records remain visible but are demoted to a legacy trust epoch and cannot supply ground truth.

Anonymized incident. In an earlier run, a stale response was attributed to the next request and poisoned the ground-truth cache. A later recomputation disagreed. Because the oracle log was append-only, the operator reconstructed the fault line-by-line, identified the first misattribution, and separated downstream claims that had consumed it. The repair added request-id and input-hash correlation, versioned the cache, and demoted every pre-fix record. No record was erased. The incident is why the oracle channel is treated as evidence that must itself be verified.

3.5 The claim funnel

Figure 1 shows five states. Findings are open observations. Breakthrough candidates have three possible promotion paths:

1. **Independent confirmation.** A different contestant has already posted a substantially overlapping claim.
2. **Adversarial critic.** A separate verifier judges the candidate against the oracle evidence it cites. This path fails closed if its external command is unavailable.
3. **Predictive match.** A preregistered oracle prediction succeeds and promotes the attached hypothesis.

A finish is a complete effective algorithm plus a correctness argument. If the challenge declares `finish_gate`, each evidence row must parse in the pinned format and matching rows must meet both the minimum count and every required coverage tag. A malformed row rejects the submission before it can enter peer verification. When no finish gate is declared, the original direct-to-peer behavior is retained. Historically, 18 of 32 finish proposals were rejected with concrete counterexamples; acceptance therefore cannot be inferred from fluency or persistence.

Only another contestant may verify a finish. Agreement causes the daemon to write `SOLVED`; rejection requires a concrete reason. Retractions and superseding proposals append new records. A stale victory marker is evidence about history and is not silently removed.

3.6 Supporting roles

The advisor is cross-vendor and reserved for hard derivations or direction checks. It never substitutes for oracle evidence. Its persistent spend ledger uses a file lock and a conservative pre-call reservation; any call that could exceed the hard dollar cap is refused before network activity. The OpenAI and Gemini SDKs are optional and imported only in the selected advisor path, so the daemon has no advisor dependency.

The optional historian periodically summarizes active threads, dead ends, and open questions. Its prompt forbids instructions: it is non-directive by construction. The manager and status views read files and health data only; they expose no mutation endpoint. The demo disables both historian and critic commands, so its retest invokes no external model or agent CLI.

3.7 Append-only reproducibility

Findings, breakthroughs, finishes, verifications, turns, and oracle events are JSONL records with human-readable mirrors. JSONL append flushes and synchronizes each line. Corrections are new records, and SOLVED is published atomically. State therefore reconstructs the research sequence, including failed predictions, rejected finishes, and trust-epoch changes.

Table 1: Arena state surfaces and mutation rules.

Surface	Writer	Reproducibility rule
<code>shared/findings.*</code>	daemon finding handler	Append one open observation; corrections add a later finding.
<code>shared/breakthroughs.*</code>	daemon promotion path	Append only after the promotion reason has been fixed.
<code>shared/finish.*</code>	daemon finish and verify handlers	Append proposal and verdict as separate records.
<code>shared/oracle_log.*</code>	daemon oracle handler	Append prediction and execution separately; retain protocol metadata.
<code>contestants/<id>/</code>	daemon continuity handlers	Journal and turns append; current direction is the sole replaceable view.
<code>round_counter.txt</code> , SOLVED	daemon only	Increment the round under lock; publish the solved record atomically.

4 Layer B: the evidence kit

The arena gates acceptance; the evidence kit gates evidence quality. It is used while preparing a finish and again after acceptance to consolidate the algorithm into checked claims. The fixed package `gauntlet/` supplies manifests, policies, gates, provenance, a sequential runner, independence lint, reports, and table generation. An exchangeable instance supplies the mathematics.

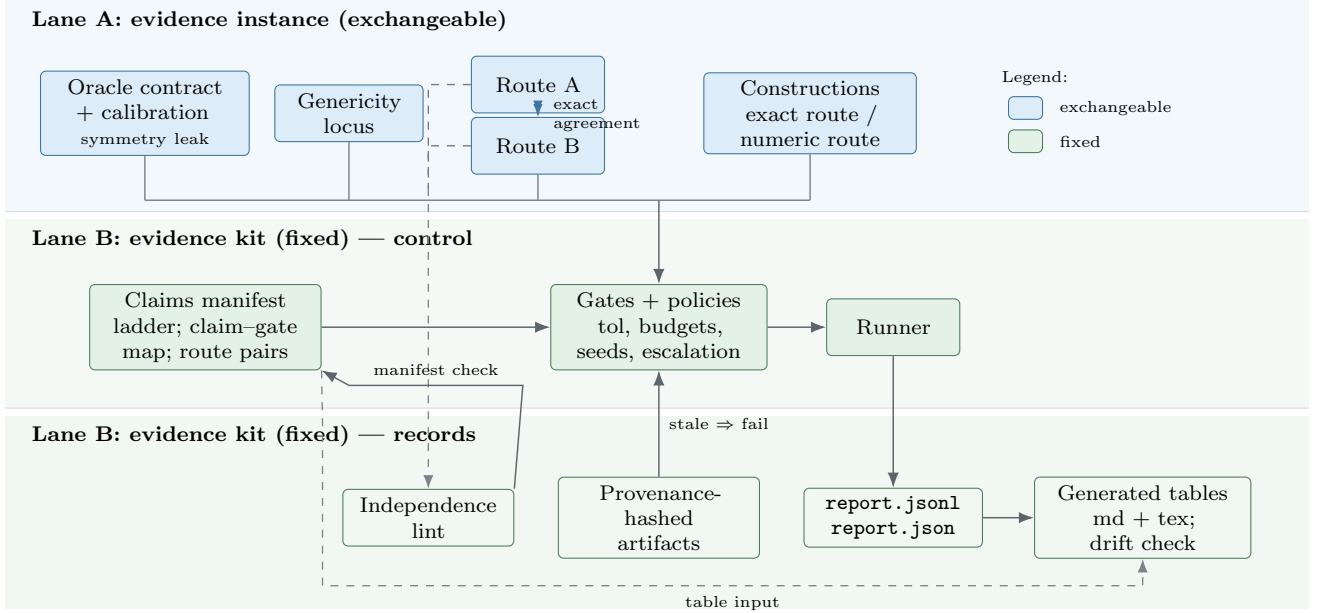


Figure 2: Layer B claim-coupled pipeline. Exchangeable reconstruction and construction modules feed fixed policies and gates. Structured records, provenance checks, import lint, and generated tables close the loop between claims, executable evidence, and the writeup.

4.1 Redundant ground truth and the claim ladder

An evidence instance reconstructs the identified object by at least two substantively independent routes. Exact objects use integer or rational arithmetic; structural facts become assertions rather than comments. The manifest declares route pairs and forbidden shared imports, while an AST lint checks direct dependencies. The lint is a mechanical backstop, not a proof of conceptual independence.

Each claim occupies one rung: `conjectured`, `numeric`, `exact`, `proved_small`, or `proved`. Promotion needs evidence appropriate to the new rung. A weakened or removed gate forces demotion to the highest still-supported rung. Proof rungs require a proof link; cheap executable checks remain valuable as regression evidence even after proof.

4.2 Gates, policy, and explicit coverage

A gate covers one claim or one tight claim group. Its policy owns tolerances, sample counts, time budgets, quick-mode scaling, size ladders, and precision stages. Random streams derive from a root seed and stable gate identifier:

$$s_g = \text{int}_{\text{big}}(\text{SHA256}(\text{text}(s) : g)[0:8]). \quad (1)$$

Equation 1 makes a gate’s stream independent of registry ordering. Root and derived seeds are recorded. Every coverage reduction is an explicit `SKIP` with a reason. A false check produces `FAIL`; an unhandled

exception becomes a failing structured check and does not prevent later gates from running.

4.3 JSONL schema

The runner writes one JSON object per line and a matching aggregate JSON object. Table 2 is the stable record surface.

Table 2: Record fields in `report.jsonl`.

Record	Fields	Purpose
meta	type, instance, root_seed, quick, started_utc	Run identity and root settings.
check	type, gate_id, name, config, verdict, criterion, measured, runtime_s, extra	One atomic probe, including measured data and error detail.
gate	type, gate_id, title, claim_ids, verdict, pass/fail/skip counts, runtime_s, seed, policy_snapshot	Gate roll-up and the exact policy used.
summary	type, ok, totals	Final run verdict and aggregate counts.

4.4 Artifacts, generated tables, and drift

An expensive artifact carries its payload, creation data, producer-code hashes, and caller metadata. Loading recomputes every declared hash. A missing producer or unequal digest marks the artifact stale, and a consuming gate fails until an explicit rebuild occurs.

The table generator joins the manifest to the latest report and emits Markdown and LaTeX. It never upgrades a claim; it makes claim-to-gate linkage visible. The drift conditions are fixed in Table 3.

Table 3: Writeup drift conditions and their meaning.

Condition	Why it is rejected
A claim names a gate missing from the report.	The documented claim has no executed evidence in this run.
An exact or proof-rung claim has no passing linked check.	The status outruns its current evidence.
A report gate is not claimed by the manifest.	Executed work is detached from the published claim surface.
Any report check has verdict FAIL .	The generated document must expose, not conceal, the failure.

5 Exchangeable demonstrations and retests

5.1 EXCHANGEABLE: Layer A demo challenge

`challenges/ising.lift/` is a worked arena package. Inputs are weighted graphs with integer edge couplings and integer vertex fields. The sealed oracle exhaustively enumerates binary sign assignments for sizes up to sixteen. It reports `wall` if any assignment is degenerate; otherwise it returns the difference between positive and negative classical counts. The contestant statement deliberately describes only the graph contract, five calibration values, and a change-of-variables leak. It withholds the Ising interpretation, which is part of the discovery task.

The finish must identify a Laurent-polynomial lift, give an effective coefficient algorithm, justify the extraction of the classical coefficient, and include at least twelve matching evidence rows. Required tags force coverage at sizes two through five, with and without fields, the deliberate leak pair, a frustrated case, a wall check, and a disconnected graph.

5.2 EXCHANGEABLE: Layer A scripted retest

`arena/smoke.arena.py` is orchestration, not a substitute contestant. It starts the daemon with its own interpreter and a temporary `ORACLE.STATE_ROOT`, then invokes the real stdlib client as `claud` and `codex`. It uses localhost only. The retest asserts the stages in Table 4.

Table 4: Deterministic arena retest coverage.

Step	Assertion surface
1	Health, served problem text, enabled oracle, and no hidden interpretation words in the contestant statement.
2	Seven pinned oracle cases, including integer and wall outputs, with request ids and input hashes.
3	A matching prediction promotes a breakthrough with reason <code>predictive_match</code> .
4	A second contestant’s overlapping finding promotes a claim through independent confirmation.
5	A finish without evidence is rejected before any verification record is created.
6	Twelve live-query rows cover every configured tag and enter peer verification.
7	The other contestant agrees; daemon health and the marker both report <code>SOLVED</code> .
8	Shared JSONL and Markdown mirrors, private continuity files, turn records, round count, and monotone line counts satisfy the state layout.
9	The daemon terminates and the temporary state root is removed.

The shipped run completes all nine stages with 219 assertions in well under two minutes and makes no external API or agent-CLI call. Because it queries the challenge oracle through the same daemon and client used by real agents, it retests both the mathematical fixture and the operational funnel.

5.3 EXCHANGEABLE: Layer B worked instance

`example_ising/` is the consolidation-side demonstration. It provides two reconstruction routes, exact and numeric construction checks, symmetry tests, a provenance-coupled finite census, and generated tables. The full stdlib run reports seven gates and 300 passing checks, with zero failures, zero skips, and a clean drift result. The specific constructions are replaceable; the manifest, gate context, runner, provenance rules, and report format remain fixed.

6 Authoring and swap contract

Layer A and Layer B are independently exchangeable.

Layer A. Copy `challenges/_template/`, rename the directory, and make `challenge.name` agree. Write the contestant task and calibration leak in `problem.md`; define schemas, promotion, and optional evidence coverage in `config.yaml`; implement sealed `query(payload)` in `oracle.py`. The template includes a commented finish-gate example.

Layer B. Copy `instance_template/`, implement two oracle routes, genericity and calibration modules, construction modules, gates, and `manifest_def.py`. Select the package with `python3 run_demo.py --instance NAME`. Any costly product consumed by a gate must use provenance-coupled artifact I/O, and the generated verification tables must be included without hand editing.

Table 5: Swap boundary for the two layers.

Layer	Exchangeable package	Fixed machinery
A: discovery	<code>problem.md</code> , <code>config.yaml</code> , <code>oracle.py</code>	daemon, client, correlated oracle protocol, state manager, promotion, historian, advisors, dashboard, status, and ops.
B: consolidation	oracle routes, calibration, genericity, constructions, gates, manifest	gate context, policy, runner, provenance, independence lint, JSONL reports, table generator, and drift check.

6.1 Layer A author checks

Before a challenge is admitted to a real contest, its author checks the boundary rather than only the answer values:

1. The directory slug and `challenge.name` agree, both schemas validate, and every configured contestant id is accepted by the client.
2. Direct oracle calls cover valid, malformed, and wall cases. The worker returns the same request metadata that the runner sent, and a deliberately altered request id or input hash is rejected.
3. Calibration contains a real symmetry-linked pair, while `problem.md` does not disclose the construction that contestants are meant to identify.
4. The finish gate, when used, has attainable but nontrivial tags. One valid matrix passes; missing rows, missing tags, malformed JSON, and a nonmatching row produce clear failures before peer review.
5. Historian and critic settings match the operating environment. Demonstrations that promise offline behavior disable both external command paths.

6.2 Layer B author checks

The consolidation package is accepted only when both routes reproduce every calibration row and seeded sweep, route lint reports no forbidden direct import, each claim names an existing gate, and every proof rung names a proof reference. Quick mode may reduce counts but must emit explicit coverage records. The full run must recreate or validate every consumed artifact, write parseable JSONL through the final summary, generate both table formats, and return a clean drift result.

The swap test is therefore behavioral. Renaming a package is insufficient: the author demonstrates that fixed code can load it, that all failure paths remain fail-closed, and that the resulting records can explain both a success and an intentional failure without hand repair.

7 Operating procedure P0–P6

P0 — Author.

Copy the Layer A template. Write the task, symmetry-linked calibration, finish criteria, schemas, promotion policy, evidence coverage, and sealed oracle. Version the package; keep runtime state and scratch data untracked.

P1 — Boot.

Start the daemon and verify health. Contestants retrieve the problem through the client and confirm the oracle mode.

P2 — Contest.

Run independent ticks. Use cross-vendor advisors only for difficult derivations, predict before informative queries, post results to the blackboard, and let configured promotion paths operate.

P3 — Finish.

Assemble the effective algorithm, correctness argument, and evidence matrix. The daemon applies the mechanical gate; the other contestant verifies or rejects with a concrete gap.

P4 — Solve.

Only the daemon, and only after peer agreement, writes **SOLVED**.

P5 — Consolidate.

Move the accepted algorithm into an evidence-kit instance. Build two routes, claim-sized gates, ladder statuses, provenance-coupled artifacts, and generated tables. Promote only what the evidence supports.

P6 — Ship.

Publish reproduction commands, generated tables, the reconstructible discovery history, and the open frontier.

8 Failure modes and defenses

Table 6: Failure modes addressed across both layers.

Failure mode	Defense
Premature victory or self-certification	Peer verification by the other contestant; daemon-set SOLVED ; stale markers retained.
Plausible but wrong claims	Three promotion paths, prediction before query, and an adversarial critic.
Groupthink or shared blind spots	Cross-vendor contestants and advisors, private construction, and stigmergic rather than conversational coordination.
Gamed or thin finish evidence	Mechanical row count and coverage tags before verification.
Oracle-channel corruption	Request id and input hash on every response, versioned cache, recomputation, and trust epochs.
Shared-state corruption or lost history	Single-writer daemon, append-only logs, and superseding corrections.
Runaway advisor spend	Fail-closed hard-cap ledger with pre-call reservation.
Context drift over long runs	Journal-first snapshots and non-directive historian digests.
Convention or sign errors in consolidation	Two independent routes, symmetry invariants, and the calibration leak.
Writeup drift from the suite	Generated verification tables and the drift check.
Silent coverage loss	Explicit SKIP records with reasons.
Stale cached artifacts	Producer hashes checked by every consuming gate.
Irreproducible randomness	Root seed, stable per-gate derivation, and recorded seeds.

9 Reproducibility surface

A complete run has two audit trails. The arena state records how the result was discovered and accepted: private continuity, shared claims, oracle traffic, finish decisions, and rounds. The evidence report records how the accepted construction was checked: meta settings, atomic checks, gate roll-ups, summary totals, artifacts, and the generated claim table. The two trails answer different questions and should not be collapsed. Peer agreement is not a proof, and a green consolidation suite does not recreate the independent discovery history.

The shipped reproduction commands are intentionally short:

```
python3 -m venv .venv
.venv/bin/pip install -r arena/requirements.txt
.venv/bin/python arena/smoke_arena.py
python3 run_demo.py
make -C report
```

The arena retest creates and removes its own state root. The evidence run is standard-library-only. The report build is separate so operational state, verification records, and typeset prose cannot silently substitute for one another.

10 Limitations

Version 0.2 does not bundle autonomous contestant runners. The operator starts and supervises each agent loop, while the daemon remains passive. Layer B executes gates sequentially; isolated contexts and derived seeds make parallel scheduling possible, but it is not implemented. Compute offload is also operator-managed: policy budgets record intent, yet no runner currently routes a heavy job to another machine.

Proof-rung claims cite human-checked proofs through `proof_ref`; there is no formal-proof integration. There is no CI configuration. The existing exit codes, JSONL records, deterministic seeds, smoke script, and build commands are suitable inputs to CI, but shipping such a wrapper would add an operational policy not required by the method.

These limits narrow automation, not the epistemic contract. The daemon still owns acceptance, the peer remains independent, the oracle channel remains correlated, evidence cuts remain explicit, stale products fail closed, and the published claim status cannot outrun its checked support.

Several boundaries remain deliberately human-facing. The finish gate checks syntax, row count, matching declarations, and tag coverage; it does not prove that a proposed value was derived honestly or that the algorithm generalizes. The peer must compare the cited rows with the shared oracle history and must inspect the correctness argument. Likewise, a passing numeric gate supports only the policy and domain it records. It cannot be described as exact evidence without a separate exact route.

The sealed worker prevents accidental imports through the normal arena path, but it is not a hostile-code container. A contestant granted unrestricted filesystem access could violate the protocol by reading challenge source. Real deployments must pair the software boundary with agent instructions or an operating-system sandbox appropriate to their threat model. The challenge author also remains trusted to implement the stated oracle contract; schema validation cannot establish that the answer key itself is mathematically correct.

Finally, direct-import lint detects obvious route coupling but not shared ideas, copied logic, or transitive dependencies. Cross-vendor roles reduce correlated error without proving independence. Review therefore remains necessary at three points: challenge authoring, peer acceptance, and any promotion to a proof rung. The method records those judgments so they can be audited; it does not pretend to automate them away.

11 Conclusion

The two layers separate discovery risk from evidence risk without separating their records. The arena makes independent agents earn a result through prediction, promotion, a mechanical evidence threshold, and adversarial peer review. The evidence kit then turns that result into reproducible, claim-sized checks with an honest proof frontier. Challenge packages and evidence instances can change; daemon sovereignty, append-only history, request correlation, independent routes, provenance, and generated tables remain the fixed method.